

NAME:KRISHITHA .G

REG NO:192524451

Course code:CSA0613

Course title: Design and Analysis of Algorithms

CO2_ASSESSMENT TOOL 2 _ SCENARIO-BASED ASSESSMENT

Title: Employee Salary Search Using Binary Search

Aim

To search an employee salary efficiently from a sorted list of salary records using the Binary Search algorithm.

Software Used

- Python

Algorithm

- Start.
- Read the sorted employee salary records.
- Read the salary value to be searched.
- Set $low = 0$ and $high = n - 1$.
- Find the middle element using $mid = (low + high) // 2$.
- Compare the middle salary with the search key.
- If both are equal, display "Salary Found" and its position.
- If the search key is smaller, search the left half by updating $high = mid - 1$.
- If the search key is larger, search the right half by updating $low = mid + 1$.
- Repeat the process until the salary is found or $low > high$.
- If the salary is not found, display "Salary Not Found".
- Stop.

Source Code (Python)

```
# Binary Search for Employee Salary Records

def binary_search(salaries, key):
    low = 0
    high = len(salaries) - 1

    while low <= high:
        mid = (low + high) // 2

        if salaries[mid] == key:
            return mid
        elif salaries[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Sorted employee salary records
salaries = [25000, 30000, 35000, 40000, 45000, 50000, 55000]
```

```
# Salary to search
key = int(input("Enter salary to search: "))

result = binary_search(salaries, key)

if result != -1:
    print("Salary found at index:", result)
else:
    print("Salary not found.")
```

OUTPUT

Output

Enter salary to search: 45000

Salary found at index: 4

=== Code Execution Successful ===

Analysis

Time Complexity Relation

At each step, Binary Search reduces the search space by half.

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Expansion

$$\begin{aligned} T(n) &= T\left(\frac{n}{2}\right) + 1 \\ &= T\left(\frac{n}{4}\right) + 1 + 1 \\ &= T\left(\frac{n}{8}\right) + 1 + 1 + 1 \end{aligned}$$

After **k** expansions,

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

When the problem size becomes 1,

$$\begin{aligned} \frac{n}{2^k} &= 1 \\ 2^k &= n \\ k &= \log_2 n \end{aligned}$$

Substituting,

$$T(n) = T(1) + \log_2 n$$

Since $T(1)$ is a constant,

$$T(n) = \Theta(\log n)$$

⋮

Documentation

- The algorithm searches the employee salary records using the Binary Search technique.
- At each step, the sorted list is divided into two equal halves, reducing the search space.
- The middle element is compared with the target salary to determine the next search direction.
- Since the search space is halved in every iteration, Binary Search is highly efficient for large sorted datasets.
- Time Complexity: $\Theta(\log n)$
- Space Complexity: $\Theta(1)$